```
  Namespace="Fruit"
  ClassName="FruitClient" />
```

In section 11.4.4 you'll learn how to customize the generated code further, but before we get to that topic, let's look at the generated `FruitClient` and how to use it.

11.4.3 Using a generated client to call your API

So far, you've been taking my word for it that a client is magically generated for your application, so in this section you get to try it out. The NSwag.ApiDescription.Client package added to your project works with the Microsoft.Extensions.ApiDescription.Client package to read the OpenAPI description file in your project. From this description it can work out what APIs you have and what types you need to serialize to and from. Finally, it outputs a C# class with the class name and namespace you specified in the `OpenApiReference` element.

NOTE The generated file is typically saved to your project's obj folder. After building your project, you can find the fruitClient.cs file in this folder. Alternatively, use Visual Studio's Go To Definition (F12) functionality on an instance of `FruitClient` to navigate to the code in your integrated development environment (IDE).

To use the `FruitClient` to call your API, you must create an instance of it, passing in the base address of your API and an `HttpClient` instance. Then you can send HTTP requests to the discovered endpoints. A client generated from the OpenAPI description of the simple minimal API in listing 11.2, for example, would have methods called `FruitPOSTAsync()` and `FruitGETASync()`, corresponding to the two exposed methods, as shown in the following listing.

Listing 11.6 Calling the API from listing 11.2 using a generated client